Prolog

Some algorithms and
Data structures

Quicksort

```
sort([],[]).
sort([P|Q],T) :- partition(P,Q,S,G),
                 sort(S,SS), sort(G,GG),
                 append(SS,[P|GG],T).
partition(P,[X|L],[X|PS],PG) :- X < P,
                                partition(P,L,PS,PG).
partition(P,[X|L],PS,[X|PG]) :- X >= P,
                                partition(P,L,PS,PG).
partition(P,[],[],[]).
```

Sieve of Eratosthenes

- Goal:
  - Find the prime numbers in the interval 1 to N

- Algorithm :
  1. Generate all integers from 2 to N
  2. Delete the multiples of the smallest numbers in the list. Repeat.
  3. Stop when the square of the current smallest number in the list is greater than the largest number in the list.

Sieve of Eratosthenes

Seules les règles 1. et 2. du crible d'Ératosthène sont appliquées ici

```
genereListe(0,[]).
genereListe(N,[N|Xs]):- N > 0,
                N1 is N-1,genereListe(N1,Xs).

retireMultiple(X,[],[]).
retireMultiple(X,[T|Q],Resultat) :- T>X,
   T mod X =:= 0, retireMultiple(X,Q,Resultat),!.
retireMultiple(X,[T|Q],[T|Resultat]) :-
   retireMultiple(X,Q,Resultat).

retireTousLesMultiples(N,[],[]).
retireTousLesMultiples(1,L,L).
retireTousLesMultiples(N,Li,L):- N>1, retireMultiple(N,Li,LL),
               N1 is N-1, retireTousLesMultiples(N1,LL,L).

premiers(N,L):- genereListe(N,Li),
            retireTousLesMultiples(N,Li,L).
```

Prolog tree

- A binary tree is a structure that can hold data. Each element of the tree contains a piece of data and has at most one ‘parent’ and two ‘children

```
        1
      /   \
    2       3
   / \     / \
  4   5   6   7
```

Prolog tree

A tree can be represented with the structure

t(elem, left, right)

where **elem** is the value associated with the node, **left** and **right** are the sub-trees (nul when there is no subtree).

Example

A tree with only one element: t(1,nul,nul)

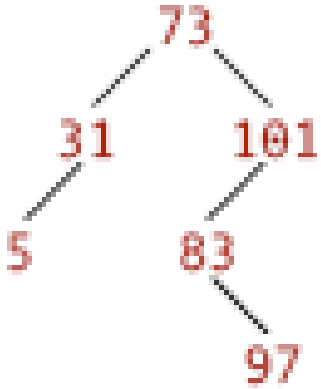
A simple tree:

```
   1
  / \
 2   3
```

t(1,t(2,nul,nul),t(3,nul,nul)).

Prolog tree

```
t(73,
  t(31,
     t(5, nul, nul),
     nul),
  t(101,
     t(83,
        nul,
        t(97, nul, nul)),
     nul))
```



Inorder traversal

```
printInfo(nul).
printInfo(t(RootInfo,LeftSubtr,RightSubtr)) :-
    printInfo(LeftSubtr),
    write(RootInfo),
    write(' '),
    printInfo(RightSubtr).
```

```
?- printInfo(t(73, t(31, t(5, nul, nul), nul),
                   t(101, t(83, nul, t(97, nul, nul)), nul))).
5 31 73 83 97 101
```

Tree search

```
search(Info, t(Info, _, _)).
search(Info, t(RootInfo, Left, _Right)) :-
        precedes(Info, RootInfo),
        search(Info, Left).
search(Info, t(RootInfo, _Left, Right)) :-
        precedes(RootInfo, Info),
        search(Info, Right).

precedes(Info1, Info2) :-
        Info1 < Info2 .
```

Tree insertion

```
insert(Info, nul, t(Info, nul, nul)).
insert(Info, t(RootInfo, Left, Right),
                t(RootInfo, LeftPlus, Right)) :-
    precedes(Info, RootInfo),
    insert(Info, Left, LeftPlus).
insert(Info, t(RootInfo, Left, Right),
                t(RootInfo, Left, RightPlus)) :-
    precedes(RootInfo, Info),
    insert(Info, Right, RightPlus).
```

Remove the root

```
delete(Info, t(Info, nul, Right), Right).
delete(Info, t(Info, Left, nul), Left).
delete(Info,  t(Info, Left, Right),
                    t(NewRoot, NewLeft, Right)) :-
    removeMax(Left, NewLeft, NewRoot).

% removeMax(Tree,NewTree,Max)
```

General node removal

```
delete(Info, t(RootInfo, Left, Right),
                t(RootInfo, LeftSmaller, Right)) :-
        precedes(Info, RootInfo),
        delete(Info, Left, LeftSmaller).
delete(Info, t(RootInfo, Left, Right),
                t(RootInfo, Left, RightSmaller)) :-
        precedes(RootInfo, Info),
        delete(Info, Right, RightSmaller).
```

Remove max element

```
removeMax(t(Max, Left, nul), Left, Max).
removeMax(t(Root, Left, Right),
                    t(Root, Left, RightSmaller), Max) :-
        removeMax(Right, RightSmaller, Max).
```